Check for updates

Received: 21 December 2021 | Revised: 3 September 2022 | Accepted: 26 September 2022

DOI: 10.1111/exsy.13158

ORIGINAL ARTICLE

Expert Systems WILEY

Expert system for automatic microservices identification using API similarity graph

Xiaoxiao Sun[1,2] | Salamat Boranbaev[1,2] | Shicong Han[1,2] | Huanqiang Wang[1,2] | Dongjin Yu[1,2]

[1]School of Computer Science and Technology, Hangzhou Dianzi University, Hangzhou, China

[2]Key Laboratory of Complex Systems Modeling and Simulation, Ministry of Education, Hangzhou, China

Correspondence
Dongjin Yu, School of Computer Science and Technology, Hangzhou Dianzi University, Hangzhou 310018, China.
Email: yudj@hdu.edu.cn

Funding information
Key Science and Technology Project of Zhejiang, Grant/Award Number: 2020C01165; National Natural Science Foundation of China, Grant/Award Number: 61472112; Natural Science Foundation of Zhejiang Province, Grant/Award Number: LQ20F020017

Abstract

As a new software design paradigm, microservices structure an application as a collection of services that are independently deployable and loosely coupled. A key step of migrating non-microservices-based systems to microservices-based systems is the identification of microservices in the target application. Traditional approaches to identify microservices, however, usually suffer from lack of full automation and low effectiveness. This paper puts forward an expert system to identify microservices automatically from legacy systems by leveraging the similarity of RESTful APIs. The system consists of three major parts. The first part calculates the candidate topic similarity and the response message similarity of APIs, and the overall similarity is obtained through their combination. Afterwards, the second part constructs a graph of API similarities with API as the node and the overall similarity as the weight. The third part employs a graph-based clustering algorithm to identify candidate microservices from the API similarity graph. Experiments conducted on open-source projects demonstrate the effectiveness of our system.

KEYWORDS
API similarity, microservices, microservices identification, migration, RESTful API

1 | INTRODUCTION

In recent decades, business has experienced growth boom in the world, which in turn led to diversification and complexity of business requirements for software products, especially for enterprise applications (Kalske et al., 2017). As a result, monolithic software systems, which have grown for years even decades, tend to be more complex and unmaintainable than ever before to meet the booming business requirements (Fritzsch et al., 2019) and the rise of new technologies such as cloud computing (Huang et al., 2021). To remain competitive in this fast-changing environment, a new architecture of software is strongly required. To address this issue, many organizations have started to migrated their existing applications from monolithic style to microservices-based architecture for the purpose of better maintainability, shorter release cycles, higher availability and better system performance (Armin et al., 2018; Mazzara et al., 2018; Ponce et al., 2019; Saidani et al., 2019). Microservices architecture is a new programming paradigm, in which the structure of an application is a collection of loosely coupled services that are deployed independently with each other and communicate with lightweight mechanisms. The benefit of this style is that it facilitates development and deployment, and scales respective services independently and concurrently by small autonomous teams. Moreover, this architecture is capable of facing the forthcoming challenge in cloud computing and cloud service (Ma et al., 2021), and the idea has also influenced design in other areas such as the Internet of Things (Butzin et al., 2016).

The very first and basic step in transferring a monolithic system into a microservice based system is microservices identification, which requires professional knowledge and profound understanding of the original application and is usually done manually with high cost of time and

 © 2022 John Wiley & Sons Ltd.

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

labour (Kecskemeti et al., 2016). Despite microservices' popularity, there still lacks disciplined understanding and consensus on the principles and activities underlying this identification (Baškarada et al., 2018; Hassan et al., 2020). Different approaches require different data source or documents to extract microservice, while some of them could be sensitive like the source codes. In other words, there is no standard process to extract microservices. Particularly, an automatic and effective identification of microservices from non-microservices-based applications with accessible input is in desperate need (Haselbock et al., 2018).

Fortunately, nowadays, many applications leverage RESTful APIs to provide functionalities to clients, which is a valid and credible data source for us to complete the task of microservice identification. Inspired by the resource-centric design principle of RESTful API (Fielding et al., 2017), this paper puts forward an approach to microservices identification based on the similarity of RESTful APIs from legacy systems. First, based on the three core principles of RESTful API, namely, resource identification, HTTP method and statelessness, we design a candidate topic similarity and a response message similarity of APIs, and the overall similarity of the APIs is obtained through their combination by considering both the APIs' semantic similarities and their connection to related resources (Xiao et al., 2021). Then, an API similarity graph is constructed with API as the node and the overall similarity as the weight. Finally, we use a graph-based clustering algorithm to identify the candidate microservices from the API similarity graph. Our approach aims to provide reference to assist architects in making decisions on microservice decomposition, sharing their pressure, shifting their focus to more detailed service design, and reducing the cost of microservice architecture transfer through automated processes.

Summarily, the main contribution of this paper is as follows: (1) we present an approach to microservices identification from non-microservices-based systems with full automation, based on the easily accessible RESTful APIs. (2) Our approach employs both the APIs' semantic similarities and their connection to related resources, leading to a more cohesive result. (3)A graph-based clustering algorithm is applied to identify candidate microservices.

The rest of this paper is structured as follows. We first review related work in Section 2. Afterwards, in Section 3, we present the preliminary to better understand the approach. Later, we show the detail of our approach in Section 4. After applying our approach on five open-source projects and one industry project as the evaluation in Section 5, we discuss the threats to the validity of our approach in Section 6. Finally, we conclude our study and outline the future work in last section.

2 | RELATED WORK

Microservices migration is a promising technique to incrementally modernize traditional monolithic enterprise applications and enable them to exploit the benefits of new technologies such as cloud-computing (Furda et al., 2018; Harms et al., 2017; Spm et al., 2019). However, decomposing legacy systems into microservices still remains a great challenge over the last years (Francesco et al., 2018).

In academia, many attempts have been made by researchers in microservices extraction from legacy systems. For example, Mazlami et al. (2017) proposed three coupling strategies (i.e., logical coupling strategies, semantic coupling strategies, and contributor coupling strategies) to extract microservices from monolithic applications, and embedded them in graph-based clustering algorithms to obtain candidate microservices. However, their coupling strategy heavily depends on the version submission history of the code. Baresi et al. (2017) proposed a method based on interface information to recognize microservices in the OpenAPI specification. Baresi et al. (2017) also presented a solution of microservices identification through interface analysis, which identifies potential candidate microservices, as fine-grained groups of cohesive operations. The limitation of these approach is that it relies on the interface solely defined by the developer, which lowers the universality of the method. Meanwhile, Amiri (2018) proposed a method of identifying microservices based on business processes. It first extracts business processes, then finds out a read/write operation of each business process to an entity to build correlation between businesses, and finally classifies and extracts candidate microservices. Daoud et al. (2020) designed a collaborative clustering-based approach to automatically identify microservices, which is also based on business processes. In 2021, Daoud et al. (2021) extended their former work by considering sematic,context and fragment dependency. The main disadvantage of the above three methods is that they all require prior knowledge to conduct the identification, which makes the automation of the transition impossible. Further, building upon existing business object and system structural definitions, Alwis et al. (2018) presented heuristics for microservices discovery by function splitting. Chen proposed a semi-automate microservices identification method driven by data flow. Instead of analysing source code, Jin et al. (2018) proposed a functionality oriented microservices extraction method by monitoring program's dynamic behaviour and clustering execution traces. Another recent approach, proposed by Zhang et al. (2020), extracts microservices from the execution and performance logs, while taking both functional and non-functional metrics into consideration. Djogic et al. (2018) followed the principle of microservice to redesign the SOA integration platform. Neural networks might be an effective tool in relation extraction of multiple microservices (Gao et al., 2021), but so far, we have found no studies using neural network on this task.

On the other hand, numerous tools and approaches have been put forward by industrial circle, focusing mainly on service discovery by experienced system architects and business experts. For example, Gabbrielli et al. (2016) developed a tool for the automatic and optimized deployment of microservices. Klock et al. (2017) created MicroADO, an open-source tool, based on the workload of microservices systems by improving its performance. Haselbock et al. (2018) investigated the importance of different areas of microservices design, and argued that the concepts for

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2021]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

partitioning an application into services have to be considered. However, after surveying 45 practitioners of legacy-to-SOA migration, Abdellatif et al. (2018) summarized that there was very little automation in industry.

In conclusion, current methods on microservices identification, both in academia and industry, rely heavily on prior knowledge of legacy systems and understanding of the business logics, and lack automation during the transformation. This inspires us to put forward an automatic approach to identify microservices from legacy systems by leveraging the similarity of RESTful APIs. Specifically, the only data source of our approach is the easily obtainable OpenAPI Specification, and no prior understanding is needed. In addition, our approach employs both the APIs' semantic similarities and their connection to related resources to conduct microservice identification, and can run with full automation.

3 | PRELIMINARIES

In this section, we present some preliminaries for better understanding of our method. An overview of the REST architecture is given first, followed by a description of the OpenAPI specification and several proposed definitions.

3.1 | Overview of REST architecture

REST (Representational State Transfer) architectural style, which was proposed for distributed hypermedia systems by Fielding, has been widely used in practice (Fielding & Taylor, 2002). It is a software architectural style that defines a set of rules for processing and transferring resource states over HTTP. These rules emphasize the scalability of interaction between components and the universality of interface, so as to meet the independent deployment and encapsulation of components. It guides developers to design and develop web system, and enhances the scalability and robustness of the system. Although REST does not depend on any network protocol, it uses HTTP protocol to transmit information since HTTP is relatively mature and there are many suitable structures such as request header to contain important identifier information. Web services that follow the style of REST architecture are called as RESTful Web services, and the programming interfaces of these services are called RESTful APIs.

The core principles of REST architecture are as follows:

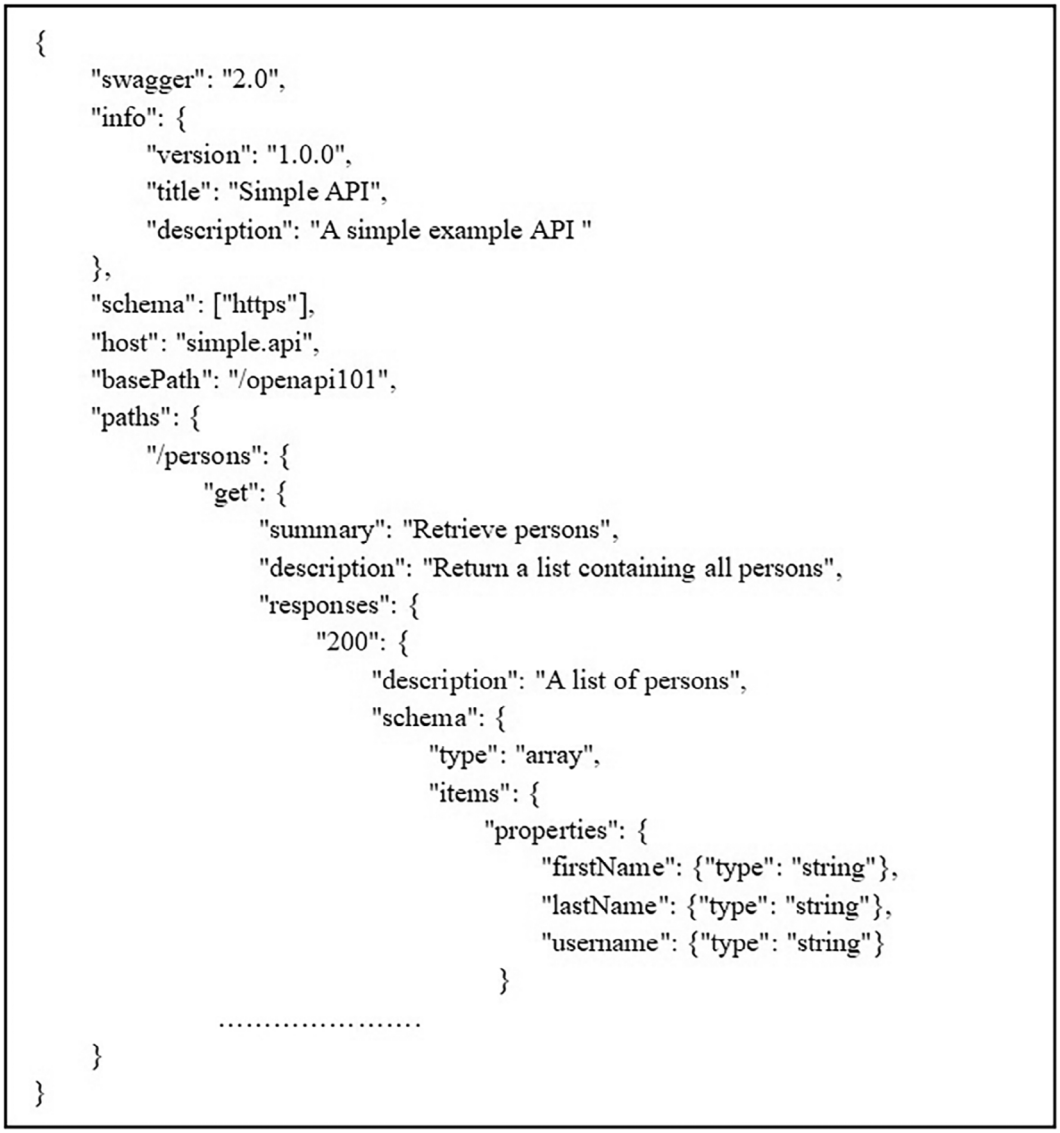
1. *Resource-oriented design*: Restful APIs are designed resource-centred, and the resources can be data, service or any kind of object that can be accessed by the client. A resource is unique and identified by an URI (Uniform Resource Identifier), which we can use to map to the related resource. For example, the URI for a specific customer could be designed as: https://application.com/customers/1.
2. *Explicitly representation of resources*: The client does not need to know the internal format and status of resources in advance. The message header of HTTP request will display the type of resources such as JSON or XML format. The client or server only needs to package the data according to the format of the display declaration. It helps to separate the clients and servers' implementations, which decouples the servers and clients and makes API request highly scalable.
3. *HTTP-based request method or verbs*: For APIs built on HTTP, performing operations on resources using standard HTTP verbs is one of the uniform interface standards. GET, POST, PUT and DELETE are the most common four operations in standard HTTP verbs.
4. *Stateless*: Stateless means that the server does not save the state of requests made by clients. HTTP requests are independent, it would be redundant for servers to save the state of requests. And each request must contain all the information to complete this operation. Information is stored in the request only. This greatly reduces the memory pressure on the server, allowing it to handle more tasks.
5. *Status Code*: The response message of the RESTful API contains both HTTP status codes and data. The HTTP status code represents the state that ended after the current HTTP operation. For example, code 200 indicates that a get method is successful. If the request data is invalid for the server, it typically returns code 400. And if the URL does not exist, the response HTTP status is code 404.

3.2 | OpenAPI specification

In the REST architectural style, the OpenAPI Specification is used as a reference to create web service descriptions. Specifically, as described by the Swagger specification,[1] the OpenAPI Specification defines a standard, language-agnostic interface to RESTful APIs which allows both humans and computers to discover and understand the capabilities of the service without access to source code, documentation, or through network traffic inspection. When properly defined, a consumer can understand and interact with the remote service with a minimal amount of implementation logic.

Figure 1 gives an example of an OpenAPI specification document, which includes document version number (i.e., the swagger field), document description (i.e., the info field), API basic information (i.e., the schemes field, host field, and base-path field) and API operation information

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



```
{
    "swagger": "2.0",
    "info": {
        "version": "1.0.0",
        "title": "Simple API",
        "description": "A simple example API "
    },
    "schema": ["https"],
    "host": "simple.api",
    "basePath": "/openapi101",
    "paths": {
        "/persons": {
            "get": {
                "summary": "Retrieve persons",
                "description": "Return a list containing all persons",
                "responses": {
                    "200": {
                        "description": "A list of persons",
                        "schema": {
                            "type": "array",
                            "items": {
                                "properties": {
                                    "firstName": {"type": "string"},
                                    "lastName": {"type": "string"},
                                    "username": {"type": "string"}
                                }
            ....................
    }
}
```

FIGURE 1 An example of OpenAPI specification document

(i.e., the paths field). In particular, API operation information records the basic information of all APIs, which is also the main data source for our approach. Specifically, each key value of the paths field represents the URI of the current API, and the HTTP request method field records the operation to the specific resource (i.e., *get* method in Figure 1). The summary field and the description field record the summary description and detailed description of the API, respectively. The responses field describes the status code (i.e., *200* in Figure 1) and content of the HTTP response message. Based on the above introduction of the API operation information, several definitions are proposed as follows in this paper.

Definition 1. Candidate Topic. *Candidate topic* is the information selected from the APIs that reflects their semantic characteristics, and directly corresponds to the resource objects. Each non-parameter element in the URI of API is considered as a candidate topic, recorded as tw_m^i. An example of candidate topics in a certain API is shown in Figure 2, where the candidate topics of this API are 'api', 'team' and 'member'. '123' is not considered as a candidate topic because it represents the teamid which is a parameter element.

Definition 2. Candidate Topic Set. *Candidate topic set* is a set of all candidate topics of an API. It is recorded as $topics_i = \{tw_1^i, tw_2^i, ..., tw_k^i, ..., tw_m^i\}$, where tw_k^i represents the kth candidate topic in the ith API.

Definition 3. Detailed Description Vocabulary Set. *Detailed description vocabulary set* is a set of words that describe the API in detail in the OpenAPI specification document, denoted as $tokens_i = \{w_1^i, w_2^i, ..., w_n^i\}$.

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

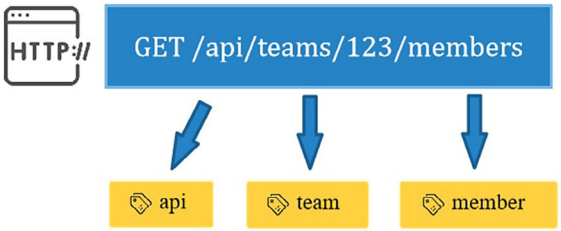


FIGURE 2 An example of *Candidate Topic*

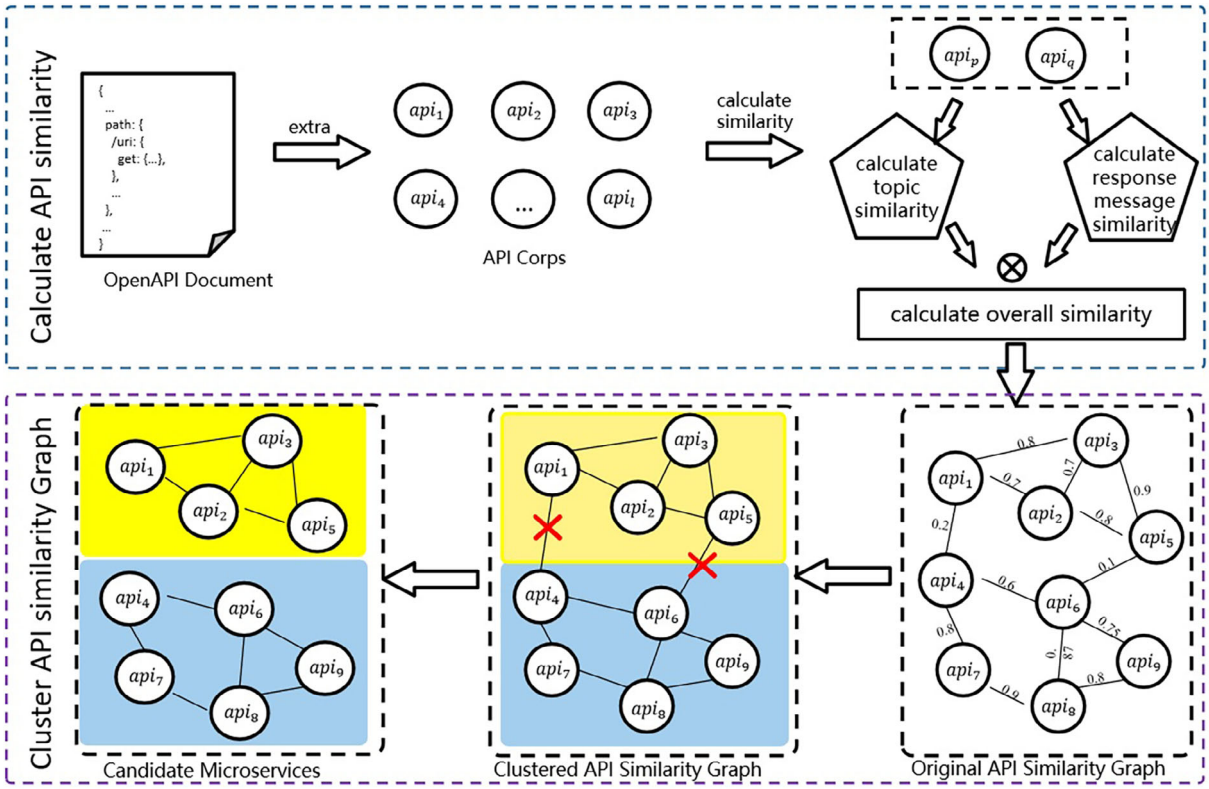


FIGURE 3 Overview of our approach

Definition 4. API Corpus: *API corpus* is a set of effective API information after data cleaning. We define API corpus as $api_{all} = \{api_1, api_2, ..., api_i, ..., api_l\}$, where api_i represents the ith API in the corpus and $api_i = \{topics_i, tokens_i\}$.

4 | APPROACH

In this section, we describe our method in detail via two main parts, that is, calculating API similarity, and clustering API similarity graph. Figure 3 depicts an overview of the main steps. The following content gives its detail.

4.1 | Calculation of API similarity

As our method take both API semantic and related resource into account, the similarity calculation consists of three parts, that is, candidate topic similarity calculation, response message similarity calculation and their synthetization as shown in Algorithm 1. We will introduce the procedure of the three parts in detail.

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 1 API similarity calculation

INPUT: two APIs: api_p, api_q
OUTPUT: Similarity of APIs api_p and api_q: $sim_{p,q}$

// Calculate topic matching degree of candidate topic
function *calc_tv*(*topic*, *api*){
    $tf \leftarrow calc_tf(topic, api)$ // TF of candidate topic
    $idf \leftarrow calc_idf(topic, api_{all})$ // IDF of candidate topic
    $ls \leftarrow calc_ls(topic, api)$ //dist. Weight of candidate topic
    return $tf * idf * ls$
}

// Calculate topic matching degree of all candidate topics
function *calc_tvec*(*api*) {
    tvec ← []
    for *topic* in *api.topics* **do**
        tvec.push(clac_tv(*topic*, *api*))
    return *tvec*
}

// Calculate the response message similarity of APIs
function $calc_resp_sim(api_p, api_q)$ {
    $sim_res_{p,q} \leftarrow \emptyset$
if $api_p.verb ==$ "*DELETE*"
        or $api_q.verb ==$ "DELETE" **do**
        $sim_res_{p,q} \leftarrow 0$
    else do
        if $api_p.type ==$ "ref" && $api_q.type ==$ "ref" **do**
            if $ref(api_p) == ref(api_q)$ **do**
                $sim_res_{p,q} \leftarrow 1$
            else do
                $sim_res_{p,q} \leftarrow 0$
            end if
        else do
            $sim_res_{p,q} \leftarrow jaccard(api_p, api_q)$
        end if
 end if
 return $sim_res_{p,q}$
}

//main routine of calculating API Similarity
$\overrightarrow{tvec_p} \leftarrow calc_tvec(api_p)$
$\overrightarrow{tvec_q} \leftarrow calc_tvec(api_q)$
// Using cosine similarity to calculate the similarity
$sim_topics_{p,q} \leftarrow calc_cosine_sim\left(\overrightarrow{tvec_p}, \overrightarrow{tvec_q}\right)$
// Calculate response message similarity of api_p and api_q
$sim_res_{p,q} \leftarrow calc_resp_sim(api_p, api_q)$
// Calculate overall similarity of api_p and api_q
$sim_{p,q} \leftarrow \alpha * sim_topics_{p,q} + (1-\alpha) * sim_res_{p,q}$
return $sim_{p,q}$

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

1. Candidate topic similarity calculation

To calculate Candidate Topic Similarity, we obtain the *candidate topic set* and *detailed description of vocabulary set* according to Definition 2 and Definition 3. Then, a *candidate topic matching degree* that indicates the importance of a candidate topic of an API is calculated by an improved TF-IDF algorithm, which gives the text similarity based on the position weight of the candidate topic in the API. The main process of calculating candidate topic matching degree is as follows.

We first calculate the *word frequency of candidate topic* by Equation (1):

$$tf\left(tw_k^i, api_i\right) = \frac{\left|tw_k^i\right|}{\left|topics_i \cup tokens_i\right|}, \tag{1}$$

where, $\left|tw_k^i\right|$ is the number of times that candidate topic tw_k^i appears in $topics_i$ of api_i, and $\left|topics_i \cup tokens_i\right|$ is the total number of words in set of $topics_i$ and $tokens_i$ of api_i. Then, the *reverse document frequency* is obtained according to Equation (2):

$$idf\left(tw_k^i\right) = \log\left(\frac{\left|api_{all}\right|}{\left|\sum_{i \in I}\left\{api_i \middle| tw_k^i \in topics_i\right\}\right|}\right), \tag{2}$$

where, $\left|\sum_{i \in I}\left\{api_i \middle| tw_k^i \in topics_i\right\}\right|$ is number of APIs where the candidate topic appears in the *API Corpus*.

Due to the fact that different positions of candidate topics in the URI and the number of times it appears as a candidate topic in the whole API corpus have different effects on their topic matching for sematic APIs, we calculate the *position weight of candidate topic* to avoid such consequences, which is based on the location of candidate topic, is designed as shown in Equation (3).

$$ls\left(tw_k^i, api_i\right) = \begin{cases} 1, & \left|N\left(tw_k^i\right)\right| - \sigma\left(tw_k^i\right) \le t \\ 1 + \frac{1}{m} * (m-k), & \left|N\left(tw_k^i\right)\right| - \sigma\left(tw_k^i\right) > t \end{cases}, \tag{3}$$

where, k represents the location of candidate topic tw_k^i in the URI of api_i, $\left|N\left(tw_k^i\right)\right|$ represents the total number of candidate topics tw_k^i as candidate topics in all APIs, m represents the number of candidate topics in api_i, σrepresents the standard deviation of the number of candidate topics, and t is a threshold value to judge whether the current candidate topic is meaningful, which is set by the user and defaults to 5% of the total number of APIs.

Finally, we calculate the *candidate topic matching degree* by combining word frequency, inverse document frequency and location weight of the candidate topic, as Equation (4) indicates.

$$tp\left(tw_k^i\right) = tf\left(tw_k^i, api_i\right) * idf\left(tw_k^i\right) * ls\left(tw_k^i, api_i\right). \tag{4}$$

We apply the Equation (4) in every candidate topic in the topic set within an API, then we can get a candidate topic vector. The *candidate topic similarity* between api_p and api_q is calculated with cosine similarity, using candidate topic vector, as shown in Equation (5).

$$sim_topics_{p,q} = \frac{\overrightarrow{tvec_p} \cdot \overrightarrow{tvec_q}}{\left\|\overrightarrow{tvec_p}\right\| * \left\|\overrightarrow{tvec_q}\right\|}, \tag{5}$$

where, $\overrightarrow{tvec_p}$ and $\overrightarrow{tvec_q}$ denote respectively the generated candidate topic vectors api_p and api_q, where each element gives the *candidate topic matching degree of* their *candidate topic*, and $\left\|\overrightarrow{tvec_p}\right\|$ and $\left\|\overrightarrow{tvec_q}\right\|$ represent modules of candidate topic vectors $\overrightarrow{tvec_p}$ and $\overrightarrow{tvec_q}$, respectively. At last, we acquire the candidate topic similarity between all APIs in *API corpus*.

2. Response message similarity calculation

In API specification document, there are two data types of response message, one is the regular type that directly shows the content of the response body (i.e., example A in Figure 4) and the other is the reference type whose response body is a reference to a certain definition (i.e., indicated by keyword of *ref* as shown example B in Figure 4). Figure 4 C gives an example of the reference. According to the above difference of response message of APIs, we calculate the *attribute similarity of response messages* between api_p and api_q as follows:

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

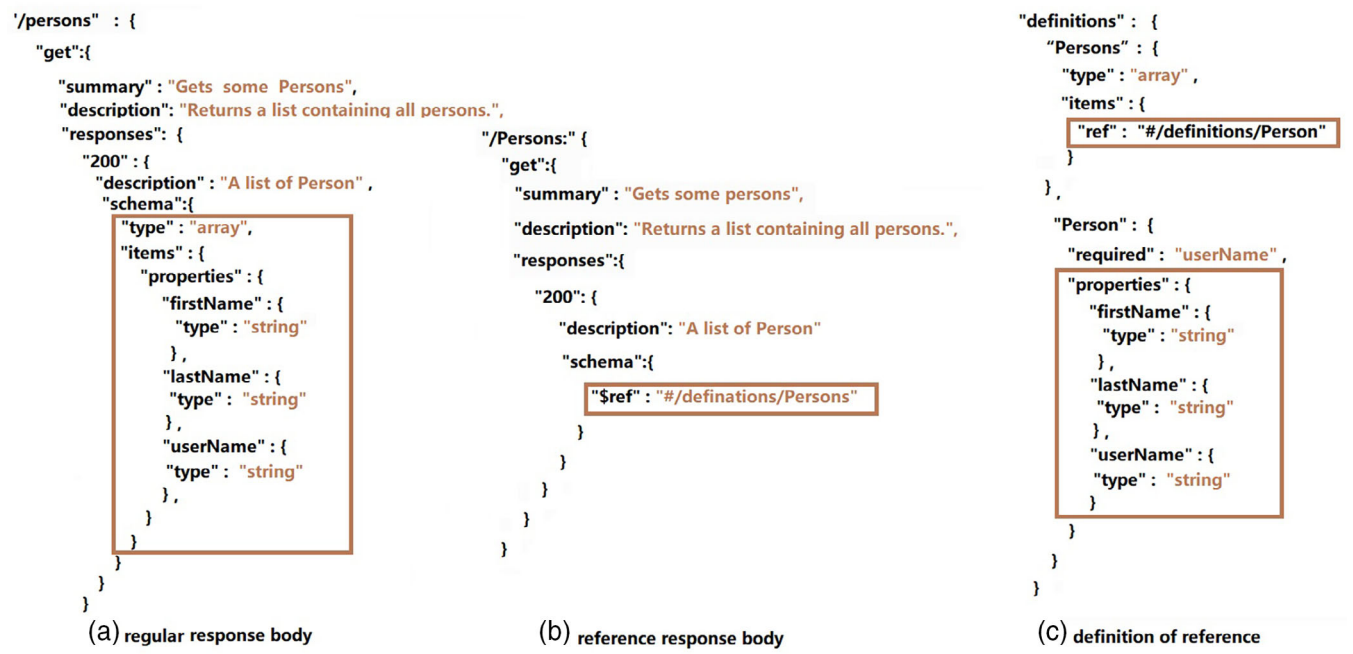


FIGURE 4 Two different data types of response message

$$sim_type_{p,q} = \begin{cases} 1, & \text{if } api_p \text{ and } api_q \text{ have } ref \text{ response types, and ultimately point to same resource} \\ 0, & \text{if } api_p \text{ and } api_q \text{ have } ref \text{ response types, and ultimately point to different resources} \\ jaccard(api_p, api_q), & \text{if neither } api_p \text{ nor } api_q \text{ has } ref \text{ response type} \end{cases} \tag{6}$$

According to Equation (6), when both api_p and api_q return the response message of type *reference*, their attribute similarity is set as 1 or 0 according to the resource location. Otherwise, the Jaccard algorithm is used to determine the attribute similarity as shown in Equation (7).

$$jaccard(api_p, api_q) = \frac{|res(api_p) \cap res(api_q)|}{|res(api_p) \cup res(api_q)|}, \tag{7}$$

where, $res(api_p)$ and $res(api_q)$ represent the attribute set of response messages of api_p and api_q, respectively.

Other than the types of response message, the difference of request methods also returns different response messages. In Figure 5, the response messages of the most common four request methods, that is, GET, POST, PUT and DELETE are displayed, which returns the resource information specified by the URI, the newly created resource object, the updated resource object and an empty set, respectively. When calculating the response message similarity between two APIs, we consider that if the request method is DELETE, the similarity of response message to any other API is 0 because a DELETE method typically returns an empty set. Otherwise, if the request method is *GET*, *POST* or *PUT*, the response message similarity is determined by $sim_type_{p,q}$.

$$sim_res_{p,q} = \begin{cases} sim_{type_{p,q}}, & \text{if request method of } api_p \text{ and } api_q \text{ if of type } GET, POST \text{ or } PUT \\ 0, & \text{if request method of } api_p \text{ or } api_q \text{ is of type } DELETE \end{cases} \tag{8}$$

3. Similarity synthetization

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

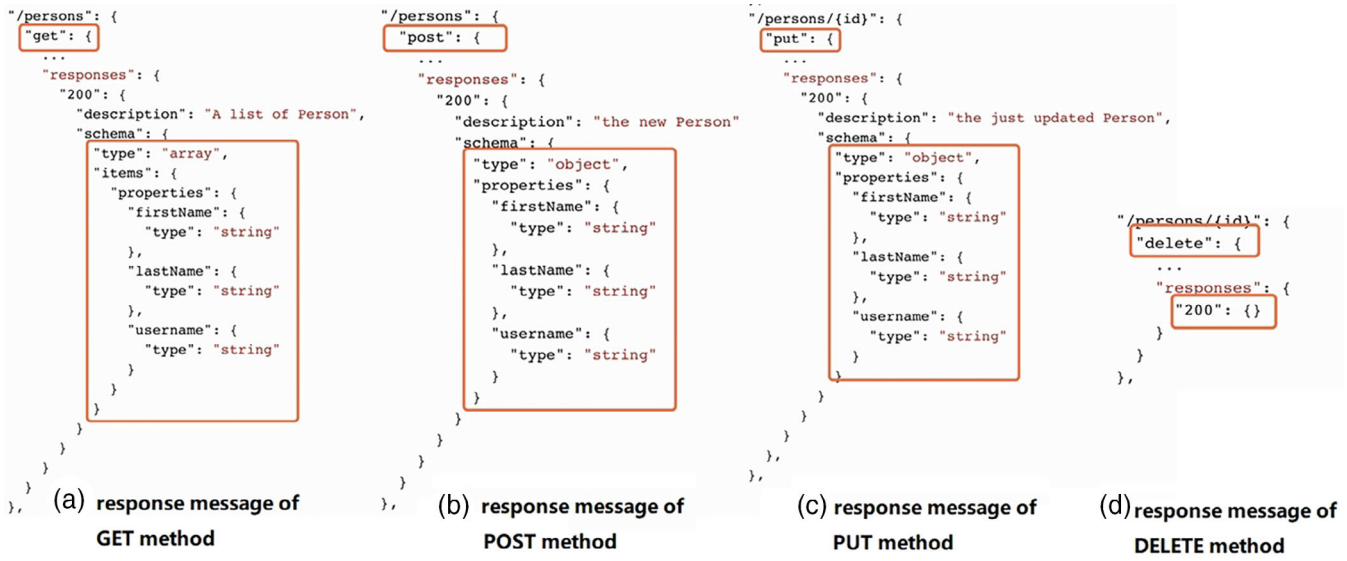


FIGURE 5 Response messages of different HTTP request methods

Finally, we synthesize the similarities of candidate topics and response messages to calculate the *overall similarity* between two APIs. When calculating the overall similarity of two APIs, we use Equation (9) to weight the two similarities.

$$sim_{p,q} = \alpha * sim_topics_{p,q} + (1-\alpha) * sim_res_{p,q}, \tag{9}$$

where, α is the weighting factor to control two kinds of similarities, and $\alpha > 0$.

ALGORITHM 2 **Microservices identification based on graph clustering**

INPUT: API similarity graph: G; number of candidate topics: h_{max}
OUTPUT: candidate microservices set: *ms*
candidate_*microservices* ← [];
// Obtain the adjacency matrix with similarity as weight
$W \leftarrow$ get_dis_matrix(G);
// Calculate degree matrix with sum of edge weight
$D \leftarrow$ get_D_matrix(W);
$L \leftarrow D - W$; // Calculating Laplace matrix
// Calculating the eigenvector of Laplace matrix
eigen_vec ← get_eigen(L);
for h in $[2, h_{max}]$ **do**
    clf ← Kmeans(h, *eigen_vec*)
    // Calculate the Calinski Harabasz score
    $score_h \leftarrow$ calinski_harabaesz(W, *clf*);
    candidate_microservices.push($score_h$, *clf*)
end for
    //The highest score is the optimal microservices partition set
    ms ← max(*candidate_microservices.score*);
    return *ms*;

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

4.2 | Clustering of API similarity graph

The clustering of API Similarity Graph also consists of three parts, that is, API similarity graph construction, graph decomposing and selection of candidate microservices as shown in Algorithm 2. We will introduce the procedure of the three parts in detail.

1. API similarity graph construction

In this section, based on the overall API similarity obtained in Section 4.1, an API Similarity Graph (ASG) is firstly constructed. In constructing ASG, APIs are applied as node and the overall API similarity are used as edges that indicate the weight of two connected nodes as shown in Figure 6.

2. Graph decomposition

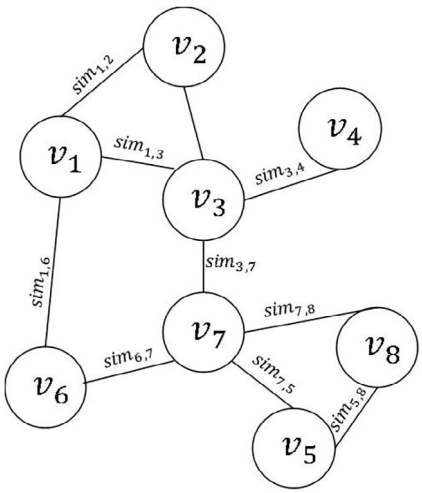


FIGURE 6 An example of API similarity graph

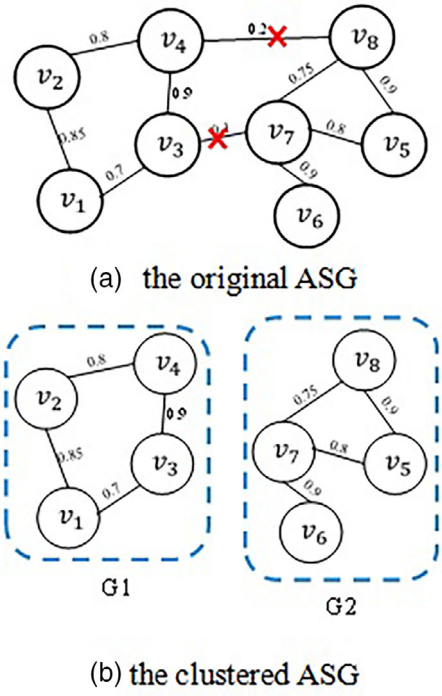


FIGURE 7 Breakdown of API similarity graph

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Afterwards, we start the graph decomposition by following the basic rules. If the weight on the edge of the two connected nodes is small, that is, the two nodes have less correlated connection, the two corresponding APIs should not belong to the same sub-graph. On the contrary, in case of high degree of correlation, two nodes should belong to the same sub-graph. After decomposition, each sub-graph is considered as a candidate microservice. Figure 7 shows an example of the process.

Specifically, the decomposition rule is derived using Normalized Cut method. First, we generate the Laplace matrix L ($L = W - D$) with the corresponding weights between all API nodes as *adjacency matrix* W and sum of weights of all edges as *degree matrix* D as shown Equation (10).

$$d_p = \sum_{q=1}^{l} sim_{p,q}. \tag{10}$$

Afterwards, we use the normalized cut method to obtain subgraphs (eigenvectors), as shown in Equation (11).

$$NCut(A, \overline{A}) = \frac{cut(A, \overline{A})}{vol(A)} + \frac{cut(A, \overline{A})}{vol(\overline{A})}, \tag{11}$$

where, $\overline{A}$ is the complement of subgraph A, $cut(A, \overline{A})$ is the loss function of subgraph A and its complement $\overline{A}$ during cutting.

Then, the sum of the weights of the edges to be truncated between subgraphs during segmentation is calculated, as shown in Equation (12).

$$cut(A, B) = \sum_{\substack{p \in A \\ q \in B}} sim_{p,q}. \tag{12}$$

Furthermore, Equation (13) gives the sum of the weights of subgraph A.

$$vol(A) = \sum_{p \in A} sim_{p,q}. \tag{13}$$

3. Selection of candidate microservices

The number of microservices cannot be determined in advance by automatic microservices identification process. Therefore, we define the total number of elements in the API candidate topic set as the largest cluster category h_{max}, and then implement the clustering process using K-means from 2 to h_{max}, respectively. To measure the performance of different clustering schemes with different number of categories h, we employ the Calinski Harabasz Index method as Equation (14). A higher score indicates a better selection of candidate microservices.

$$score_h = \frac{tr(B_h)}{tr(W_h)} * \frac{l - h}{h - 1}, \tag{14}$$

where, l is the total number of APIs in the API corpus, B_h is the covariance matrix between the subgraphs in the subgraph segmentation scheme, W_h is the covariance matrix of the internal data of the subgraphs in the graph decomposition, and tr is the trace of the covariance matrix.

5 | EXPERIMENTS AND DISCUSSION

In this section we present four different experiments to assess and validate our approach.

TABLE 1 Parameters of experiments

Parameters	Values
Weighting factor α	[0.1]
Threshold t	5%*ApiCount
Clustering scoring function	Calinski Harabasz Index
num k in K-means clustering	[2, h_{max}], h_{max} equals to number of candidate topics

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

5.1 | Experiment settings

Our experiments were run on a 10 core Intel(R) Core (TM) i9-7900X CPU @ 3.30 GHz with 64 GB RAM. Besides, models were implemented in Java on the Intellij IDEA platform, and the jdk version is 1.8. Several detailed parameters of the experiments are shown in Table 1.

5.2 | Decomposition of monolithic application

In this section, we expound the first experiment we preformed, that is, microservices identification for a well-known monolithic system called Cargo Application,[2] and we compare the performance of our approach to the method of Service Cutter, which is a state-of-art tool for service decomposition (Gysel et al., 2016). The main purpose of the application is to deliver a cargo to some place. A cargo is identified by a unique tracking id, and it always has an origin and a route specification. Delivery represents the actual transportation of the cargo, as opposed to the customer requirement (RouteSpecification) and the plan (Itinerary). A handling activity explain how and where a cargo can be handled, and can be used to express predictions about what is expected to happen to a cargo in the future. Route specification describes where a cargo's origin and destination are, and the arrival deadline. The domain model and expected decomposition of Cargo Application is shown in Figure 8.

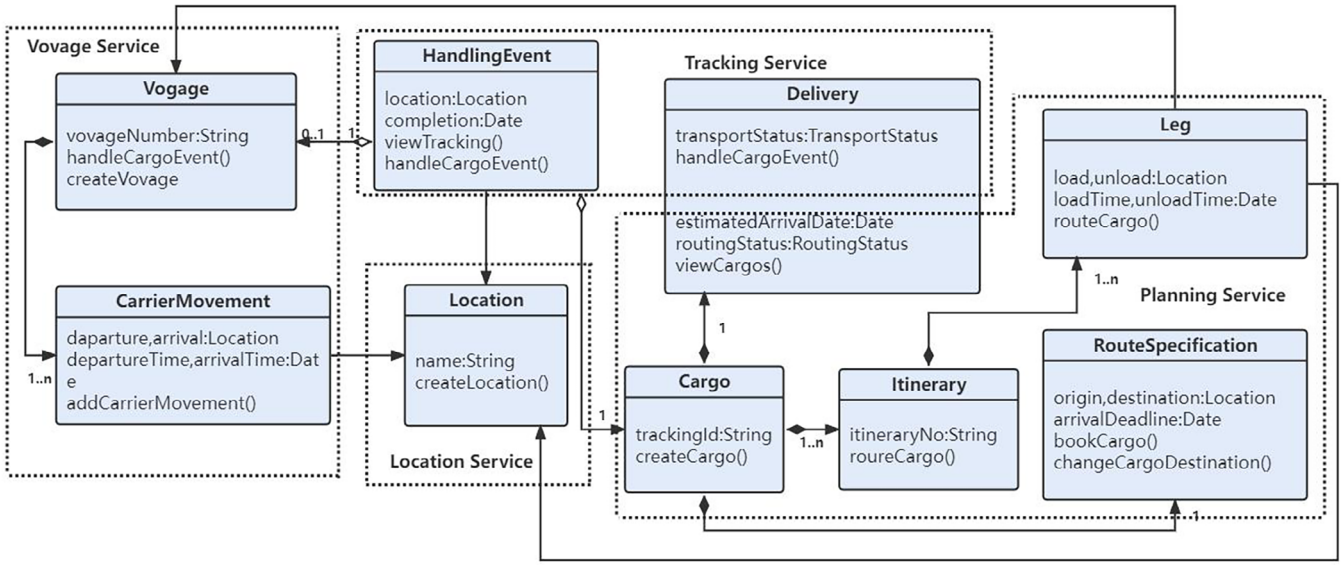


FIGURE 8 Domain model and expected decomposition of Cargo Application

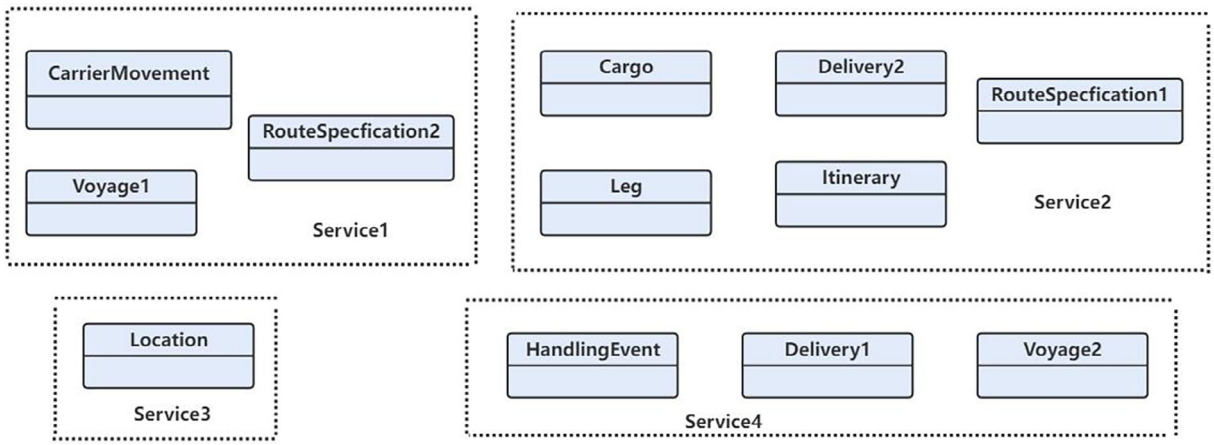


FIGURE 9 Decomposition result of our approach

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2021]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

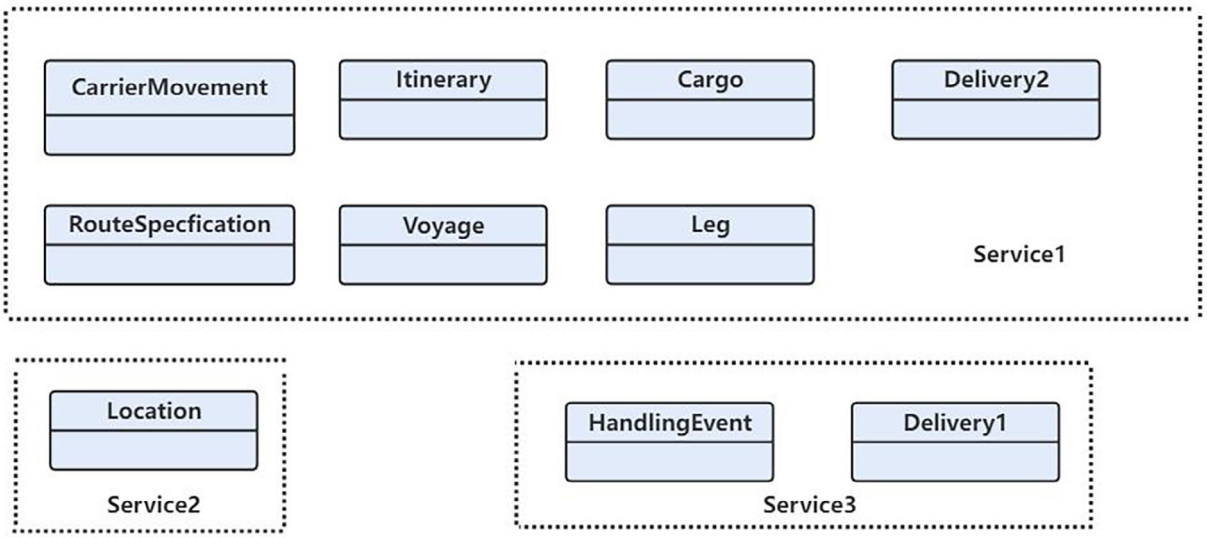


FIGURE 10 Decomposition result of Service Cutter

As present in Section 4, our approach takes OpenAPI specification of the application (i.e., Scheme.org) as input and returns a set of candidate microservices, and the result is presented in Figure 9. Service Cutter, on the other hand, takes software engineering artefacts associated with coupling criteria, such as use cases，sequence diagram and class diagram as input, and output an undirected and weighted graph, in which nodes in the graph indicate a candidate microservices and a cluster method is applied to aggregate the nodes to form the ultimate services. Figure 10 is the decomposition result of Service Cutter. From a comparative perspective, the input of our is widely accepted and can be easily generated from other documents while the input to Service Cutter is much more difficult to obtain in a unified standard of document.

As for the decomposition result, from comparison of Figure 8 with Figures 9 and 10, no approach returns the exact same result as the domain model. We can see that our approach generated four microservices while Service Cutter generated three microservices, which are different both in number and in detail. In the result of Service Cutter, the number of artefacts in a service can be very different while in our approach, they are relatively divided equally into each service, which is more consistent to the domain model. In particular, the decomposition of Service 1 and Service 2 of our approach is more similar to Voyage Service and Planning Service in the domain model than Service 1 of Service Cutter.

Despite the above discussion, we still cannot tell which approach is better since none of them reaches the expected decomposition results since no quantitative metrics can be used to evaluate the performance in a same level, which inspired us to conduct the second experiment in Section 5.2.

5.3 | Decomposition of microservice application

Considering that it would be subjective to judge if our approach has the optimal performance in monolithic applications merely by descriptions as Section 5.1, we conducted the second experiment in which all five applications we used for are microservices architecture in this section, and the original service division can be applied as a gold standard for performance evaluation of different methods. In this way, several objective indicators, such as the precision, recall, accuracy et.al of the decomposition can be calculated and compared.

5.3.1 | Evaluation metrics

To evaluate the effectiveness of our approach, we conduct extensive experiments on five open-source projects from GitHub including *Kanban*,[3] *Money Transfer*,[4] *Piggy Metrics*,[5] *Microservices Event Sourcing (MES)*,[6] and *Sock Shop*.[7] We selected them as the target projects because they are microservices-based with different sizes. We first downloaded the projects and created the OpenAPI documents based on the APIs they provide. Afterwards, we use our approach to identify microservices based on their RESTful APIs and see how the identified microservices fit the original ones in these projects. To obtain exact evaluation, we divide identified microservices into three types, that is, PMS, GMS and BMS. It is worth noting that GMS are also acceptable, and is considered as reasonable divided ones together with PMS.

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

1. *Perfect microservices (PMS)*: The microservices identified by our approach are completely consistent with the original microservices in the project;
2. *Good microservices (GMS)*: The microservices identified by our approach are consistent with the original microservices in the project after simple composition/decomposition;
3. *Bad microservices (BMS)*: The microservices identified by our approach include the resource objects of other microservices.

Taking into account the aforementioned types, the precision and recall indicators are calculated as follows:

$$Precision = \frac{|TP|}{|TP + FP|}, \tag{15}$$

$$Recall = \frac{|TP|}{|FN + TP|}, \tag{16}$$

where, $|TP|$ represents the number of reasonably divided microservices in the total number of PMS and GMS, $|FP|$ represents the number of all wrongly identified microservices, that is, BMS, $|FN|$ represents the number of correct microservices in the original application that are not identified by our approach.

Besides, we use *API division accuracy* as the metric that calculates the accuracy through the correctly identified API, as shown in Equation (17):

$$Accuracy = \frac{|API_{right}|}{|API_{all}|}, \tag{17}$$

where, API_{right} and API_{all} represent the number of correctly divided APIs, and the total number of APIs.

5.3.2 | Experimental results

Table 2 shows the experimental results for the five target applications. Due to the limited page length, only part of results is given. In addition, we analyse the performance of all applications in detail in the following context.

As shown in Figure 11, the original *Kanban* application includes four microservices: *Board Operation*, *Board Query*, *Task operation* and *Task query* while our approach identified two microservices: *Board* and *Task*. However, when analysing *Board Operation* and *Board Query* microservices, we find that both microservices operate on board resources. These two microservices are separated based on the CQRS (Command Query Responsibility Segregation) model, which is an architectural pattern that separates the models for reading and writing data. Similarly, task resources are also divided into two corresponding microservices in the original project. As for our approach, since there is no further decomposition based on CQRS, we combine two microservices, that is, *Board Operation* and *Board Query* with the same resource into one microservice of *Board*. The same situation exists for *Task Operation* and *Task Query* as *Task* service. Since after simple decomposition we could reach the correct identification, we consider our division reasonable and mark these two services as GMS.

Figure 12 depicts the experimental results of the *Sock Shop* project. We find that our approach exactly identifies four microservices, that is, *Order*, *Catalogue*, *Card* and *Payment*, which are consistent with the original project. However, the original *User* microservice is identified as four microservices in our approach (i.e., *User*, *Card*, *Addr*. and *Other*), among which the first three contain API operations towards the resources of user, card, and address, respectively. This fully conforms to the idea of our approach and these three microservices are reasonably divided and belong to GMS. Nevertheless, the fourth identified microservice (i.e., *Other*) includes API get/login and API post/register, which belong to *User* operation, and is thus marks as BMS in the result.

As for the other three projects, our approach also achieves good performance, that is, the identified microservices rarely contain BMS. In addition, we find that the granularity of the microservices identified by our approach is mostly finer than that of the original project as we can see in Table 2. The only exception is the *Kanban* project that we analysed above.

5.3.3 | Comparison with existing methods

In order to further measure the effect of the proposed approach, we compare it with two different approaches, which have similar thought to our approach and also conduct microservices identification based on the interface information from the OpenAPI specification. The first approach we

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 Part of experimental results of microservices identification on five open-source projects

Project	Identified services	Applied restful APIs	Result type
Kanban	Board	Post/api/boards Get/api/boars Get/api/boards/{id}	GMS
	Task	Post/api/tasks Delete/api/task{id} Put/api/tasks/{id} Put/api/tasks/{id}/backlog Put/api/tasks/{id}/complete Put/api/tasks/{id}/history Put/api/tasks/{id}/schedule Put/api/tasks/{id}/start Get/api/tasks	GMS
Money Transfer	Account	Post/api/accounts Delete/api/account/{id} Get/api/account/{id} Get/api/account/{id}/history Get/api/account/{id}/accounts	GMS
	Customer	Post/api/customers Get/api/customers Get/api/customers/{id} Post/api/transfer	BMS
	Other	Delete/api/customers/{id}/toaccounts/{id} Post/api/customers/{id}/toaccounts	GMS
MES	Account	Get/v1/accounts Get/v1/accounts/{id}/orders	GMS
	Catalogue	Get/v1/catalogue	GMS
	Inventory	Get/v1/inventory	GMS
	Order	Post/v1/orders Get/v1/orders/{id}	GMS
	Shopping Cart	Get/v1/cart Post/v1/cart/checkout	GMS
	User	Post/v1/users Get/v1/users/{id} Put/v1/users/{id} Delete/v1/users/{id}	GMS
	Other	Put/v1/users/{id} Get/v1/inventory/product/{id}	BMS
Piggy Metrics	Account	Post/accounts Get/accounts/{name}	GMS
	Notification	Put/notification/settings/current Get/notification/settings/current	PMS
	Statistics	Get/statistics/{account} Get/statistics/current Put/statistics/{account}	PMS
	Account1	Get/accounts/current Put/accounts/current	GMS
Sock Shop	Order	Get/orders Post/orders	GMS
	Catalogue	Get/catalogue Get/catalogue/{id} Get/catalogue/size Get/tags	GMS
	Users	Get/customers Delete/customers/{id} Get/customers/{id} Get/customers/{id}/carts Get/customers/{id}/addresses	GMS

(Continues)

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 (Continued)

Project	Identified services	Applied restful APIs	Result type
	Cart	Get/carts{customerId} Delete/carts/{customId} Post/carts/{customerId}/{items} Delete/carts/{customerId}/items/{id}	GMS
	Payment	Get/health Post/paymentAuth	GMS
	Card	Get/cards Post/cards Delete/cards/{id} Get/cards/{id}	GMS
	Addr.	Get/addresses Post/addresses Delete/addresses/{id} Get/addresses/{id}	GMS
	Other	Get/login Post/register	BMS

compared is a microservice decomposition approach based on semantic similarity (Baresi et al., 2017). It decomposes the system by matching the terms used in the specification with the reference vocabulary to identify potential candidate microservices. The second one is also a semantic-similarity-based decomposition approach (Al-Debagy & Martinek, 2020). It extracts operation names in the specification, and uses a well-trained word embedding model to convert them into vector space. Then an Affinity Propagation algorithm is applied to cluster similar operation into groups.

The comparative results of precision and recall of the three methods are given in Figure 13, where the horizontal coordinate indicates five different projects, and the vertical coordinate shows the score of precision and recall on these projects. It can be seen that our approach reaches either best or equally good precision in all projects. In particular, in *Kanban* and *Piggy Metrics* applications, whose system structures are relatively simple, all three methods archive great result (i.e., 1.0 of both precision and recall). As for more complex applications such as *Sock Shop* and *MES*, the performance of our approach outperforms other approaches. In *Money Transfer* application, however, the recall of comparative approach1 achieves 100%, which is the highest among all methods. The reason is that in this application, service *Transaction* contains only one API and only Baresi's approach successfully identified the service, while both our approach and Al-Debagy's approach failed to identify this service. As we can see in the result of Table 2, our approach wrongly identified this API in service *Customer* as BMS. But since Baresi's approach wrongly divided a customer API into service *Account*, its precision is hence lowered to a value that is the same as the other approach.

Figure 14 compares the API division accuracy between our approach and the comparative approaches, where the first column is the optimized result by changing the weight factor α, the latter two columns are the result of comparative approaches, and the last column is the result of our approach while the weight factor α set to 1, meaning that only semantic similarity is considered during the decomposition. It can be seen that our approach (i.e., the first column) achieves the best or equally best accuracy in all projects. The reason is our approach comprehensively considers the resources and returning values of API operations, while the two comparative approaches identify microservices only by the name of API interface operations. In case of only considering semantic similarity (i.e., the fourth column), our method also achieves better results in most applications since our method takes into account the weight of the token's impact on the overall semantics due to the order while other methods take every token as equally important.

5.3.4 | Analysis of weighting factor α

The weighting factor α is a parameter to balance the similarity of candidate topics and response messages in the calculation of API overall similarity. Different weighting factors may affect the experimental results.

In this experiment, the values of weighting factor α are set to 1, 0.75, 0.5, 0.25, and 0, respectively. Among them, weighting factor $\alpha = 1$ indicates that only candidate topic similarity is considered, and weighting factor $\alpha = 0$ indicates that only response message similarity is used. From Figure 15, we can see that when $\alpha = 0.75$, that is, when considering both candidate topic and response message similarities but having more weight on the former, the experimental results of all datasets archive the best. Therefore, we choose $\alpha = 0.75$ as an optimal setting.

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

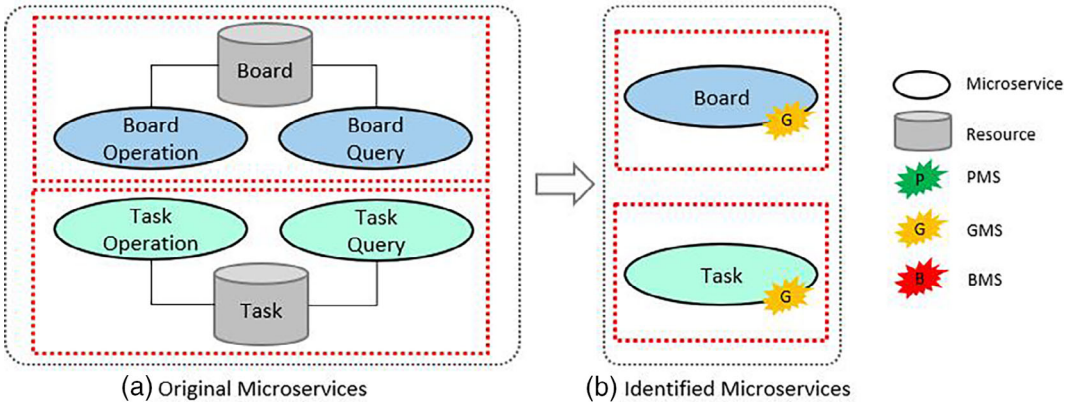


FIGURE 11 Microservices identification of *Kanban*

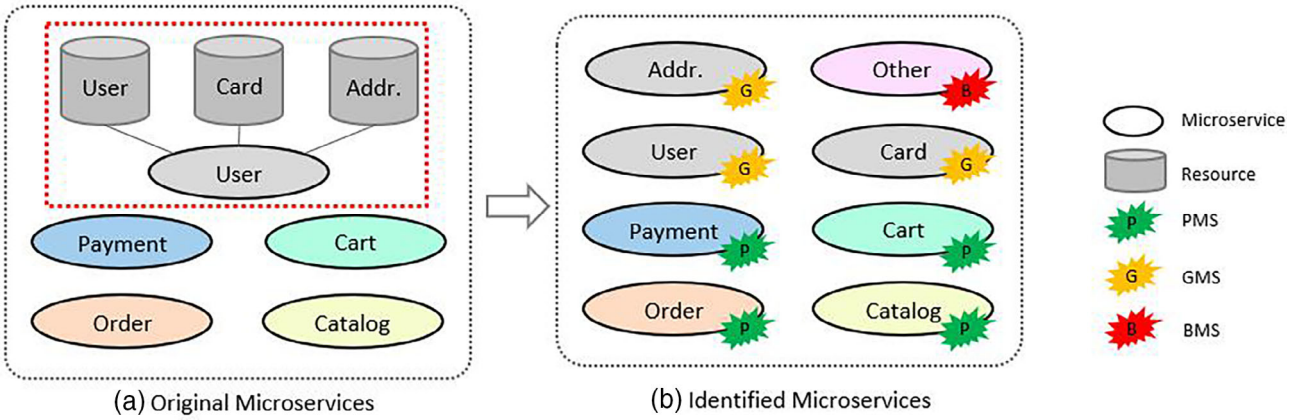


FIGURE 12 Microservices identification of *Sock Shop*

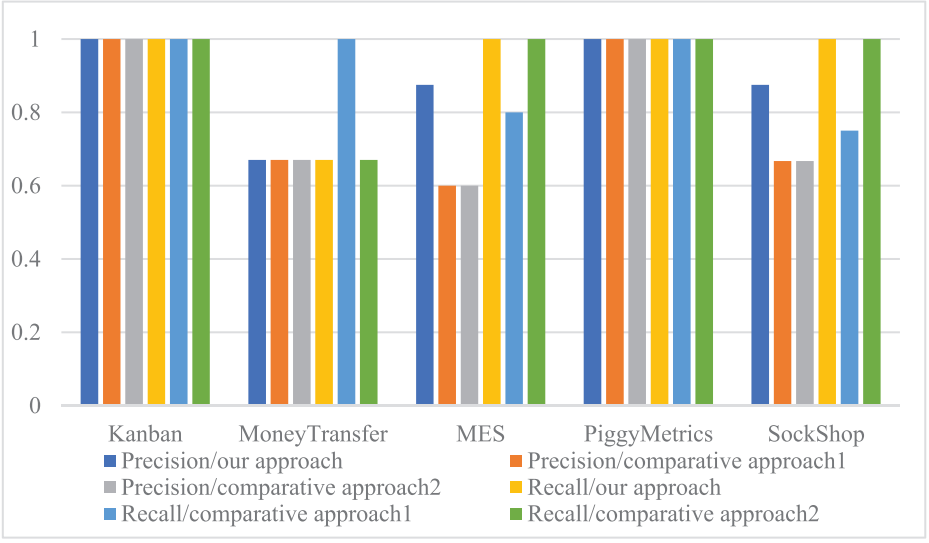


FIGURE 13 Comparison of precision and recall

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

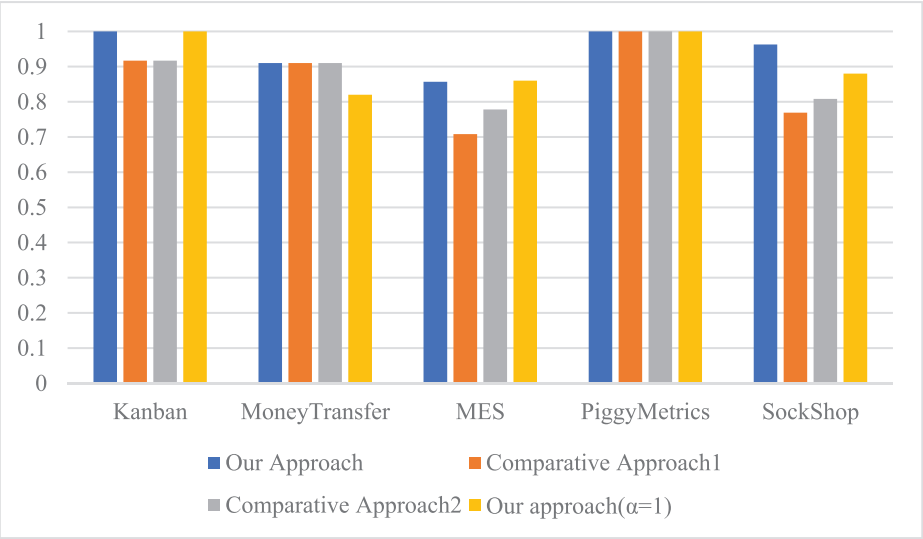


FIGURE 14 Comparison of API division accuracy

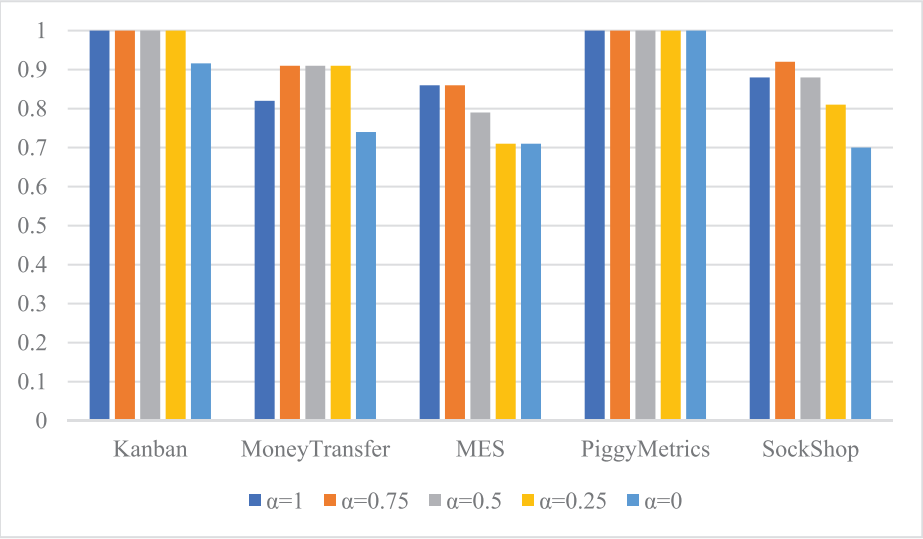


FIGURE 15 Influence of weighting factor α on division accuracy

6 | CONCLUSION AND FUTURE WORK

In conclusion, in this paper, we present an approach to automatically identify microservices from non-microservices-based systems based on the easily accessible RESTful APIs. It employs both the APIs' semantic similarities, and their connection to related resources, which leads to a more cohesive and authentic result. Extensive experiments on open-source projects demonstrate its effectiveness.

Although the proposed approach based on API similarity achieves satisfactory results, there are still some shortcomings. Specifically, when calculating the similarity of candidate topics, we adopt an improved method based on TF-IDF combined with the description of Restful APIs. Due to the fact that the essence of such practice is based on word frequency, in the cases of using different words to represent the same topic, our approach would identify the result as multiple topics. In the future, we plan to calculate topic matching degree based on topic semantics to solve this problem. Besides, we also plan to conduct more experiments to evaluate the effectiveness of the approach in the industrial field.

ACKNOWLEDGEMENTS

The work is supported by Natural Science Foundation of China (No. 61472112), Natural Science Foundation of Zhejiang Province (No. LQ20F020017) and the Key Science and Technology Project of Zhejiang (No. 2020C01165).

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

CONFLICT OF INTEREST

The authors declared that they have no conflicts of interest to this work.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in MsDecomposer at https://github.com/HduDBSI/MsDecomposer.

ENDNOTES

[1] http://swagger.io.

[2] http://github.com/citerus/dddsample-core (Java implementation).

[3] https://github.com/eventuate-examples/es-kanban-board.

[4] https://github.com/cer/event-sourcing-examples.

[5] https://github.com/sqshq/PiggyMetrics.

[6] https://github.com/chaokunyang/microservices-event-sourcing.

[7] https://github.com/microservices-demo/microservices-demo.

REFERENCES

Abdellatif, M., Hecht, G., Mili, H., Elboussaidi, G., Moha, N., Shatnawi, A., Privat, J., & Guéhéneuc, Y.-G. (2018). State of the practice in service identification for soa migration in industry. In *International conference on service-oriented computing* (Vol. 11236, pp. 634–650). LNCS.

Al-Debagy, O., & Martinek, P. (2020). Extracting microservices' candidates from monolithic applications: Interface analysis and evaluation metrics approach. In *2020 IEEE 15th international conference of system of systems engineering (SoSE)* (pp. 289–294). IEEE.

Alwis, A. D., Barros, A., Polyvyanyy, A., & Fidge, C. (2018). Function-splitting heuristics for discovery of microservices in enterprise systems. In *International conference on service-oriented computing* (pp. 37–53).

Amiri, M. J. (2018). Object-aware identification of microservices. In *IEEE international conference on services computing* (pp. 253–256). IEEE.

Armin, B., Abbas, H., Pooyan, J., Tamburri, D. A., & Theo, L. (2018). Microservices migration patterns. *Software: Practice and Experience*, *48*, 2019–2042.

Baresi, L., Garriga, M., & Renzis, A. D. (2017). Microservices identification through interface analysis. In *European conference on service-oriented & cloud computing* (pp. 19–33). Springer, Cham.

Baškarada, S., Nguyen, V., & Koronios, A. (2018). Architecting microservices: Practical opportunities and challenges. *Journal of Computer Information Systems*, *1-9*, 428–436.

Butzin, B., Golatowski, F., & Timmermann, D. (2016). Microservices approach for the internet of things. In *2016 IEEE 21st international conference on emerging technologies and factory automation (ETFA)* (pp. 1–6). IEEE.

Daoud, M., El Mezouari, A., Faci, N., Benslimane, D., Maamar, Z., & El Fazziki, A. (2021). A multi-model based microservices identification approach. *Journal of Systems Architecture*, *118*, 102200.

Daoud, M., Mezouari, A. E., Faci, N., Benslimane, D., Maamar, Z., & Fazziki, A. E. (2020). Automatic microservices identification from a set of business processes. In *International conference on smart applications and data analysis* (pp. 299–315). Springer.

Djogic, E., Ribic, S., & Donko, D. (2018). Monolithic to microservices redesign of event driven integration platform. In *2018 41st international convention on information and communication technology, electronics and microelectronics (MIPRO)* (pp. 1411–1414). IEEE.

Fielding, R., & Taylor, R. N. (2002). Principled design of the modern web architecture. *ACM Transactions on Internet Technology*, *2*(2), 115–150.

Fielding, R. T., Taylor, R. N., Erenkrantz, J. R., Gorlick, M. M., Whitehead, J., Khare, R., & Oreizy, P. (2017). Reflections on the REST architectural style and principled design of the modern web architecture (impact paper award). In *Joint meeting on foundations of software engineering* (pp. 4–14). ACM.

Francesco, P. D., Lago, P., & Malavolta, I. (2018). Migrating towards microservice architectures: An industrial survey. In *2018 IEEE international conference on software architecture (ICSA)* (pp. 9–38). IEEE Computer Society.

Fritzsch, J., Bogner, J., Wagner, S., & Zimmermann, A. (2019). Microservices migration in industry: Intentions, strategies, and challenges. In *2019 IEEE international conference on software maintenance and evolution* (pp. 481–490). IEEE.

Furda, A., Fidge, C., Zimmermann, O., Kelly, W., & Barros, A. (2018). Migrating enterprise legacy source code to microservices: On multi-tenancy, statefulness and data consistency. *IEEE Software*, *35*, 63–72.

Gabbrielli, M., Giallorenzo, S., Guidi, C., Mauro, J., & Montesi, F. (2016). *Self-reconfiguring microservices* (Vol. 9660, pp. 194–210). Springer International Publishing, LNCS.

Gao, H., Xu, K., Cao, M., Xiao, J., Xu, Q., & Yin, Y. (2021). The deep features and attention mechanism-based method to dish healthcare under social IoT systems: An empirical study with a hand-deep local–global net. *IEEE Transactions on Computational Social Systems*, *9*(1), 336–347.

Gysel, M., Kölbener, L., Giersche, W., & Zimmermann, O. (2016). Service cutter: A systematic approach to service decomposition. In *European conference on service-oriented and cloud computing* (pp. 185–200). Springer, Cham.

Harms, H., Rogowski, C., & Iacono, L. L. (2017). Guidelines for adopting frontend architectures and patterns in microservices-based systems. In *Joint meeting on foundations of software engineering* (pp. 902–907). ACM.

Haselbock, S., Weinreich, R., & Buchge He, R. G. (2018). An expert interview study on areas of microservice design. In *2018 IEEE 11th conference on service-oriented computing and applications (SOCA)* (pp. 137–144). IEEE.

Hassan, S., Bahsoon, R., & Kazman, R. (2020). Microservice transition and its granularity problem: A systematic mapping study. *Software Practice and Experience*, *2*, 1651–1681.

Huang, Y., Xu, H., Gao, H., Ma, X., & Hussain, W. (2021). SSUR: An approach to optimizing virtual machine allocation strategy based on user requirements for cloud data center. *IEEE Transactions on Green Communications and Networking*, *5*(2), 670–681.

Jin, W., Liu, T., Zheng, Q., Cui, D., & Cai, Y. (2018). In *2018 IEEE international conference on web services (ICWS) – Functionality-oriented microservice extraction based on execution trace clustering* (pp. 211–218). [IEEE 2018 IEEE international conference on web services (ICWS) – San Francisco, CA, USA (2018.7.2–2018.7.7)].

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Kalske, M., Mkitalo, N., & Mikkonen, T. (2017). *Challenges when moving from monolith to microservice architecture* (pp. 32–47). Springer.

Kecskemeti, G., Marosi, A. C., & Kertesz, A. (2016). The ENTICE approach to decompose monolithic services into microservices. In *Proceedings of the 2016 international conference on high performance computing simulation (HPCS)* (pp. 591–596). IEEE.

Klock, S., Van Der Werf, J. M. E. M., Guelen, J. P., & Jansen, S. (2017). Workload-based clustering of coherent feature sets in microservice architectures. In *2017 IEEE international conference on software architecture (ICSA)*. IEEE.

Ma, X., Xu, H., Gao, H., & Bian, M. (2021). Real-time multiple-workflow scheduling in cloud environments. *IEEE Transactions on Network and Service Management*, *18*(4), 4002–4018.

Mazlami, G., Cito, J., & Leitner, P. (2017). Extraction of microservices from monolithic software architectures. In *IEEE international conference on web services* (pp. 524–531). IEEE.

Mazzara, M., Dragoni, N., Bucchiarone, A., Giaretta, A., Larsen, S. T., & Dustdar, S. (2018). Microservices: Migration of a mission critical system. *IEEE Transactions on Services Computing*, *14*(5), 1464–1477.

Ponce, F., Márquez, G., & Astudillo, H. (2019). Migrating from monolithic architecture to microservices: A rapid review. In *2019 38th international conference of the Chilean computer science society (SCCC)* (pp. 1–7). IEEE.

Saidani, I., Ouni, A., Mkaouer, M. W., & Saied, A. (2019). Towards automated microservices extraction using muti-objective evolutionary search. In *International conference on service-oriented computing* (pp. 58–63). Springer.

Spm, A., Cyf, A., Yc, A., Ihl, A., & Cwl, B. (2019). Graph-based and scenario-driven microservice analysis, retrieval, and testing. *Future Generation Computer Systems*, *100*, 724–735.

Xiao, J., Xu, H., Gao, H., Bian, M., & Li, Y. (2021). A weakly supervised semantic segmentation network by aggregating seed cues: The multi-object proposal generation perspective. *ACM Transactions on Multimedia Computing Communications and Applications*, *17*(1s), 1–19.

Zhang, Y., Liu, B., Dai, L., Chen, K., & Cao, X. (2020). Automated microservice identification in legacy systems with functional and non-functional metrics. In *2020 IEEE international conference on software architecture (ICSA)* (pp. 135–145). IEEE.

AUTHOR BIOGRAPHIES

Dr. Xiaoxiao Sun is currently a lecturer at Hangzhou Dianzi University. She received the Ph.D. degree from Zhejiang University, Hangzhou, China in 2017. Her current research interests include spatio-temporal data mining, business process management, intelligent software engineering, and so forth.

Salamat Boranbaev was a graduate student in Hangzhou Dianzi University. His research interest includes intelligent software engineering.

Shicong Han is a master student in Hangzhou Dianzi University. His research interests include business process management, intelligent software engineering, and so forth.

Huanqiang Wang was a graduate student in Hangzhou Dianzi University. His research interest includes intelligent software engineering.

Dr. Dongjin Yu is currently a professor at Hangzhou Dianzi University, China. His research efforts include intelligent software engineering, service computing, and data engineering. He is the director of Big Data Institute, and the director of Computer Software Institute of Hangzhou Dianzi University. He is a senior member of IEEE, and a senior member of China Computer Federation (CCF). He also serves as the executive members of the technical committees of Software Engineering and Service Computing CCF.

How to cite this article: Sun, X., Boranbaev, S., Han, S., Wang, H., & Yu, D. (2022). Expert system for automatic microservices identification using API similarity graph. *Expert Systems*, e13158. https://doi.org/10.1111/exsy.13158

14680394, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.13158 by Universitaet Luebeck, Wiley Online Library on [06/11/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License